

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

Organisation of Islamic Cooperation (OIC)

Board Bazar, Gazipur, Bangladesh

Department of Computer Science & Engineering (CSE)

FINAL PROJECT REPORT

KIDSCRIBE

A Tablet Application that Teaches Children Aged 5 to 8 to Write by Hand

Course Title: Software Project Lab-II

Course Code: SWE 4404

Academic Year: 2024-25

Group 18

Hasan Mahi — 230042129

Zuhaaer Mustafa Sadab — 230042132

Md Shahib Al Kabir — 230042157

Supervised by

Farzana Tabassum

Department of Computer Science & Engineering

Submission: 1 November 2026

Table of Contents

1. Executive Summary
2. Problem Domain and Background
3. Literature and Existing Solutions
4. Requirements (SRS Session)
5. System Design (Design Session)
6. Implementation (Implementation Session)
7. Tools, Version Control, Testing, and CI/CD
8. Professional, Ethical, and Legal Considerations
9. Project Management and Feasibility
10. Teamwork and Individual Contributions
11. Results, Limitations, and Future Work
12. Glossary and References

Front Matter

Field	Detail
Project title	KidScribe — a tablet application that teaches children aged 5 to 8 to write by hand
Course	SWE 4404: Software Project Lab-II, Academic Year 2024-25
Department	Computer Science and Engineering (CSE), Islamic University of Technology
Supervisor	Farzana Tabassum, Department of CSE
Team	Group 18 — Hasan Mahi (230042129), Zuhaaer Mustafa Sadab (230042132), Md Shahib Al Kabir (230042157)
Source repository	https://github.com/Kabirohi/kidscribe-spl2-project
Platform	Android tablet and phone (Flutter 3.x / Dart 3.x)

1. Executive Summary

Learning to write by hand is still one of the first hard things a child is asked to do at school, and it is one of the few school skills that cannot be learned by watching. A child has to hold a pencil, form each letter in a particular order, and repeat it until the hand remembers. In a class of forty, a teacher cannot stand beside every child while that happens, and at home most parents are not sure which part of a letter their child is getting wrong.

KidScribe is an Android tablet application that sits in that gap. A child writes on the screen with a stylus and the app marks the writing immediately, the way a teacher leaning over the desk would: it says whether the whole letter was covered, whether the pen stayed on the line, and whether the strokes were made in the order children are taught. The child works through five stages — practice strokes, then all 26 letters, then whole words, then sentences — and a stage only opens when the one before it is finished.

The application works without an internet connection, because the schools we have in mind cannot rely on one. Handwriting is read by a recognition model that runs on the tablet itself, and every child's progress is stored on the device. Parents and teachers get their own side of the application: a parent can add several children, set a daily reminder, photograph a toy so its name becomes a word to practise, and read a weekly progress report. A teacher can create classes, see at a glance which students are falling behind, give homework or a timed test with no help on screen, and export a report for a parents' meeting.

The work was carried out across the three SPL-II sessions. The SRS session produced 24 functional requirements and 33 user stories; the Design session produced a three-layer architecture; and the Implementation session produced the working application. **All 33 user stories are implemented.** The code is 97 Dart files and roughly 19,800 lines, it passes the Dart analyser with no warnings, and 129 automated test cases covering 58 of the 80 unit tests in our SRS run in about 23 seconds. The remaining 22 tests need a physical device and are run by hand against a written checklist.

What a reader should take away is that the project is finished rather than demonstrated: the application builds, installs and runs end to end, the requirements are traceable from the stakeholder need to the test that proves it, and the places where we fell short — a continuous integration pipeline that is planned but not built, a payment screen that is deliberately a demonstration, and a synchronisation design that never deletes — are written down in Section 11 rather than hidden.

2. Problem Domain and Background

2.1 Domain, Stakeholders, and Motivation

The domain is early-years handwriting instruction, for children roughly between five and eight years old. At that age a child is learning letter formation: not only what a letter looks like but the sequence of pen movements that produces it. Teaching that sequence matters, because a child who draws an 'A' as three unconnected marks in an arbitrary order will write slowly for years afterwards. Handwriting teaching therefore cares about something most drawing software ignores entirely: the order and direction of strokes.

Three groups of people use the system, and they want different things from it.

Stakeholder	What they need	How KidScribe answers it
Children (5-8)	To practise letters without an adult beside them, and to be told immediately and kindly what to fix.	Stylus practice with instant marking, a tip that names the mistake, stars, badges and a daily streak.
Parents	To know whether their child is improving, and to keep them practising.	Child profiles, a weekly progress report with weak letters, a daily reminder, writing challenges, and words made from the child's own toys.
Teachers	To see a whole class at once and find who needs help.	Classes with a per-student progress view, a 'needs help' marker, homework and timed tests, and exportable reports.

A deliberate decision follows from the age group: **children do not have accounts.** Only parents and teachers sign up. A grown-up adds a child profile or a student and chooses who is practising, and the application stores that

child's work under them. This keeps credentials away from young children and keeps a shared classroom tablet usable by a whole class.

2.2 Fundamental Computer Science Concepts Underlying the Problem (CO1)

The problem is not solved by any single clever trick; it decomposes into several standard computer-science problems, and naming them is what made the implementation tractable.

- **Data structures.** A letter is a list of strokes, and a stroke is an ordered list of points: `List<List<Offset>>`. Progress is a map from an item identifier to a score; earned badges are a set, because order is irrelevant and membership tests must be fast. Choosing a set rather than a list for badges, and a nested list rather than a flat one for strokes, is what later makes the stroke-order check possible at all.
- **Computational geometry.** Marking a trace is a nearest-neighbour problem in two dimensions. We compute coverage (what fraction of the template was reached) and precision (what fraction of the ink landed on the template), both using a Euclidean distance threshold.
- **Resampling and curve parameterisation.** To compare two strokes drawn at different speeds with different numbers of points, both are resampled to a fixed number of points spaced evenly along arc length. This is what makes stroke direction detectable.
- **Coordinate normalisation.** Every shape is stored in a unit square, 0 to 1, rather than in pixels, so the same data is correct on any screen and the scoring tolerance is a fraction of the canvas rather than a fixed pixel count.
- **Persistence and serialisation.** The device store holds only primitive types, so each progress map is encoded as a JSON string under a composite key and decoded on read.
- **Design patterns.** The observer pattern (through BLoC streams), the repository pattern (services that hide where data lives), dependency injection through constructors, and immutable value objects compared by value.
- **Finite progression logic.** The five stages form a simple state machine: the current stage is the first one not yet complete, and later stages are locked.

2.3 Specialist Knowledge Required (CO1)

Four areas of knowledge outside ordinary application programming were necessary, and the project would have produced a worse result without each of them.

- **Handwriting pedagogy.** Letters are taught in families of similar shape rather than alphabetically, and each letter has a designated stroke order and direction. We encoded both: the 26 letters are grouped into families, and every template records its strokes in teaching order with a green start point and an orange end point.
- **On-device machine learning.** Free-hand writing has no guide to compare against, so shape geometry alone is insufficient. We use Google ML Kit Digital Ink Recognition, which reads the sequence and timing of strokes rather than a finished image, and which runs entirely on the device so that practice works offline.
- **Stylus input handling.** A child resting a hand on a tablet generates touch events that would otherwise be drawn as scribble. We implement palm rejection: once a stylus event is seen, finger events are ignored for the rest of the session.
- **Interface design for young children.** Large targets, very few words, pictures instead of labels where possible, and feedback that names one thing to fix rather than listing faults. Error messages are written to be read aloud by a parent to a five-year-old.

3. Literature and Existing Solutions

Tracing applications for children are a crowded category, and we reviewed the most widely used examples before writing the SRS, together with the technical sources that informed the marking algorithm.

Existing solution	What it does well	What it does not do	KidScribe's position
LetterSchool	Polished animation and strong motivation for letter shapes.	Guided tracing only; the child is rarely asked to write unaided, and there is no teacher view.	We keep guided tracing but require a second, unguided attempt before a letter counts as learned.
Writing Wizard	Configurable letter sets and rewards; allows custom words.	Scores proximity to the outline; does not enforce the order or direction of strokes.	We score the manner of writing as a separate, equally binding check.
iTrace	Designed with teachers; records data for review.	Requires an account and network connection; little support for one shared tablet.	We work fully offline and switch learner with one tap on a shared device.
Montessori-style apps	Follow established pedagogy, including letter families.	Usually fixed vocabulary, no personalisation to the individual child.	We let a parent add words by photographing the child's own toys.

On the technical side, the marking approach draws on standard work in online handwriting recognition, where a stroke is treated as a time-ordered sequence rather than an image, and on the common practice of arc-length resampling before comparing two curves. Google's ML Kit Digital Ink documentation informed the decision to supply stroke timing as well as position.

How our problem statement differs. Every reviewed product answers the question 'does this look like the letter?'. None of the consumer products answers 'was this written the way it is taught?', and none combines that with a classroom view on a device that works without internet. KidScribe's problem statement is therefore narrower and more specific: mark the *manner* of writing, not only the result, and do it offline on one shared tablet.

4. Requirements (SRS Session)

4.1 Functional Requirements and Stakeholder Traceability (CO2)

The SRS session produced 24 functional requirements and 33 user stories. The table below traces a representative requirement from each functional area back to the stakeholder need that motivated it and forward to the test that proves it; the complete matrix of 24 requirements, 33 user stories with acceptance criteria, and 80 unit tests is in `/doc/srs.pdf`.

FR	Requirement	Stakeholder need	US	Test
FR-1	Parents and teachers can create an account	Grown-ups need their own space; children must not hold credentials	US-1, 2, 3	T_1-T_11
FR-5	Children practise basic strokes with a stylus	Children need hand control before letters	US-8	T_23, T_24
FR-7	The system checks each letter and gives instant feedback	A child alone needs to be told what to fix	US-10	T_27, T_28
FR-9	Children build sentences and learn basic grammar	Writing is more than letters	US-12, 13	T_31-T_34
FR-10	Five learning stages are guided in order	Difficulty must rise gradually	US-14	T_35, T_36
FR-14	Badges, streaks and rewards	Children need a reason to return	US-18	T_43, T_44
FR-16	Parents add words using the toy camera	Children practise words they care about	US-20	T_47, T_48
FR-18	Teachers get a class view of progress	A teacher must find who is behind	US-22, 23, 24	T_52-T_58
FR-20	Data is stored on the device and synced across devices	Schools have unreliable internet	US-26, 27	T_61-T_64

FR	Requirement	Stakeholder need	US	Test
FR-24	A timed test with no help on screen	Teachers need unaided evidence of ability	US-33	T_77-T_80

4.2 Non-Functional Requirements

NFR	Requirement	How it is met
NFR-01	A result appears within about 5 seconds	Marking is local arithmetic; measured well under one second. ML recognition adds roughly a second on device.
NFR-02	Practice works without internet	All templates, scoring and storage are local; the recognition model is downloaded once and then runs offline.
NFR-03	Large buttons and few words	A shared widget set with a minimum 48dp target, pictures in place of labels, and messages written for a five-year-old.
NFR-04	Each learner's work is kept separate	Every storage key is scoped to the active child or student.
NFR-05	The application must not lose progress	Only improvements overwrite a stored score, and synchronisation merges rather than replaces.

4.3 Contextual and Sustainable-Development Considerations

Three contextual factors shaped the requirements. **Connectivity:** primary schools in Bangladesh frequently have no usable internet, which is why offline operation is a requirement rather than a feature. **Device cost and lifetime:** the target is one shared mid-range Android tablet per class, not a device per child, so the application must switch learner quickly and must run on modest hardware; keeping recognition on the device also avoids the recurring cost and energy of server inference. **Equity:** because a shared tablet is the realistic deployment, nothing in the child-facing flow requires a login, a subscription, or a parent's phone number. In sustainability terms the application extends the useful life of existing hardware rather than requiring new purchases, and its offline design means no continuous data transfer for routine use.

4.4 SRS Session Feedback and the Changes Made

Three pieces of feedback from the SRS session changed the document materially.

Feedback received	Change made
Authentication was described in general terms; the reviewer noted that industry practice is token-based and asked us to be specific.	The login requirement was rewritten around token authentication, and the implementation moved to Firebase Authentication, which issues a signed ID token and refreshes it automatically.
The requirement set did not say what happens after a grown-up reviews a child's work.	FR-22 was added for written feedback on a profile or on a finished letter (US-30).
Assigned work had no time dimension, so a task could sit open indefinitely.	FR-23 was added for a time limit on an assigned task, covering both a teacher's deadline (US-31) and a parent's challenge (US-32), and FR-24 was added for a timed test that hides all on-screen help and submits itself when the time expires (US-33).

These additions took the SRS from 21 functional requirements and 29 user stories to **24 functional requirements, 33 user stories and 80 unit tests**. We also rejected several suggestions that would have contradicted FR-1 by giving children their own logins; those are recorded in the SRS as out of scope.

5. System Design (Design Session)

5.1 System Architecture

KidScribe uses a three-layer architecture, implemented with the BLoC pattern. The constraint that defines it is that dependencies point in one direction only.

Layer	Location	Responsibility	May call
1. Presentation	`lib/screens`, `lib/widgets`	Draws the interface and forwards user gestures. Holds no rules.	Layer 2 only
2. Business logic	`lib/blocs`	Holds all application state and every rule. Decides what is true and publishes it.	Layer 3 only
3. Data	`lib/services`	Device storage, Firebase, the ML models, file export. Knows nothing of the interface.	External systems

A screen never calls a service directly, and no service imports anything from a screen. That single rule is what makes the business layer testable without a device, and it is why 58 of the 80 unit tests can run on a desktop with no emulator attached.

5.2 Component and Data Models

The data classes exchanged between layers are small, immutable value objects. The principal ones, each traceable to a requirement, are:

Model	Fields	Traces to
`UserModel`	email, password, name, role	FR-1, FR-2
`ChildProfile`	id, name, age	FR-3
`SchoolClass`	id, name, students	FR-4
`TraceItem`	id, glyph, name, hint, strokes	FR-5, FR-6
`TraceResult`	score, stars, passed, coverage, precision, orderTip	FR-7
`FreehandResult`	percent, passed, recognizedAs, usedMl, kidMessage, grownUpReport	FR-6, FR-7
`Assignment` / `AssignItem`	id, title, kind, items, inOrder, dueAtMs, testMinutes, doneKeys, grade	FR-17, FR-18, FR-23, FR-24
`FeedbackNote`	id, target, text, fromRole, atMs	FR-22
`ScopeSummary`	strokesDone, lettersDone, wordsDone, totalChecks, lastPractice	FR-19

Storage schema. There is no relational database. The device store is a key-value store holding one JSON document per concern, and every key carries a *scope* identifying whose progress it is — the teacher's selected student, otherwise the parent's active child, otherwise the account itself.

```
trace_progress_<scope>   {"s1_standing": 3, "s1_sleeping": 2}
letters_v2_<scope>       {"A": {"guided": 3, "free": 82}}
words_v1_<scope>         {"CAT": 1}
sentences_v1_<scope>     {"s01": 1}
games_v1_<scope>         {"match_best": 4, "ninja_best": 2}
practice_log_<scope>     {"2026-10-28": 12}
assignments_<scope>      [ {id, title, kind, items, dueAtMs, ...} ]
feedback_<scope>         [ {id, target, text, fromRole, atMs} ]

Cloud (Firestore)  kidscribe_sync/{uid}   {key: {'t': type, 'v': value}}
```

5.3 Design Alternatives Considered and the Trade-Off Made

Decision	Chosen	Alternative rejected	Why
State management	BLoC with flutter_bloc	Plain setState inside each screen	setState was our first implementation. It put rules inside widgets, so nothing could be tested without rendering. BLoC separates rules from drawing; the cost is more files, which we accepted for testability.

Decision	Chosen	Alternative rejected	Why
Handwriting recognition	On-device ML Kit Digital Ink	A cloud recognition API	A cloud call is more accurate but needs connectivity and has a per-call cost, which contradicts NFR-02 and the deployment context. On-device costs nothing per use and works offline; we accept slightly lower accuracy and a one-off 20 MB model download.
Local storage	SharedPreferences with JSON documents	SQLite via sqflite	Our data is a handful of small documents read whole, with no queries or joins. SQLite would add a schema, migrations and a dependency for no benefit at this size. The cost is that we cannot query across children without loading everything, which Section 11 records as a limitation.
Synchronisation	Cloud Firestore, merge-on-conflict	Last-write-wins replacement	Losing a child's stars is the worst failure the product can have, so the merge keeps the larger of two numbers and the union of two collections. The accepted cost is that deletions do not propagate.

5.4 Design Session Feedback and the Changes Made

Feedback received	Change made
The design did not follow a recognised state-management pattern; the reviewer asked for an explicit architecture rather than logic held in widgets.	The whole application was migrated to the BLoC pattern: 20 feature folders under `lib/blocs`, a full Bloc with named events for authentication and Cubits elsewhere. The decision and its one exception are documented in ARCHITECTURE.md.
Documents used language that was too technical for the intended audience.	The design report and all user-facing text were rewritten in plain language, with each technical term explained where it first appears. The same standard is applied in this report.
Rapid pen input would be pushed through the state layer, which the reviewer flagged as a likely performance problem.	Pen strokes are deliberately kept in a local controller at roughly 60 points per second and handed to the bloc only when the child presses Check. The exception is documented rather than left implicit.

6. Implementation (Implementation Session)

6.1 Codebase Structure and File Tree

The repository follows the departmental submission standard, with the application under `/code` and the documents under `/doc`. Generated folders (`build/`, `.dart_tool/`) are excluded below and from version control.

```
kidscribe-spl2-project/
|-- code/
|   |-- lib/                      97 Dart files, ~19,800 lines
|   |   |-- screens/      30      what the user sees (layer 1)
|   |   |   |-- learn/            the five learning stages
|   |   |   |-- games/            Letter Match, Letter Ninja, Free Writing
|   |   |   +-- assign/           giving tasks, grading, feedback
|   |   |-- blocs/        29      rules and state (layer 2), 20 feature folders
|   |   |-- services/     15      storage, Firebase, ML Kit (layer 3)
|   |   |-- widgets/      12      shared pieces: writing pad, mascot, park scene
|   |   |-- models/        4      plain data classes
|   |   |-- data/          3      letter templates, word list, sentence list
|   |   |-- theme/         2      colours and text styles
|   |   |-- main.dart             entry point; wires the three layers
|   |   +-- firebase_options.dart
|   |-- unit_testing/
|   |   |-- test/         15      129 automated test cases
|   |   |-- results/              saved output of the last run
|   |   +-- KidScribe_Unit_Testing.pdf
|   |-- assets/
|   |   |-- art/         120      Microsoft Fluent Emoji (MIT)
|   |   +-- sounds/        1      success chime
|   |-- tool/                     SVG-to-rig converter and source artwork
|   |-- android/                  Android build configuration
|   |-- pubspec.yaml              packages and asset declarations
|   +-- ARCHITECTURE.md           the three layers, explained
|-- doc/
|   |-- srs.pdf                   24 FRs, 33 user stories, 80 unit tests
|   |-- design.pdf                architecture, models, alternatives
|   |-- final-presentation.pdf    the deck presented on 28 October 2026
|   +-- final-report.pdf          this document
+-- README.md                     how to build, run and test
```

Why it is organised this way. A folder corresponds to a layer, not to a feature, so the path of a file states what it is permitted to touch: anything in `screens` may call a bloc but not a service. Inside `blocs` there is one folder per feature, so everything about one feature's state is in one place. Plain data and templates live in `models` and `data` and import nothing from the application, which is why the tests can use them directly.

6.2 Communication Between Components and the Data Exchanged

There are four boundaries in the system, and each exchanges a specific, documented shape.

Boundary	What crosses it	Format
Screen → Bloc	A method call with plain arguments	`cubit.check(pad.strokes)`
Bloc → Screen	One immutable state object, compared by value	`BadgesState(loading, earned, streak, best, practicedToday)` extending Equatable, so an unchanged state causes no redraw
Bloc ↔ Service	A model in, a result object out	`TraceResult(score, stars, passed, coverage, precision, orderTip)`
Service ↔ device store	One JSON string per key	`letters_v2_<scope> -> {"A": {"guided": 3, "free": 82}}`
Service ↔ Firestore	A type-tagged map per value	`{'t': 's'|'i'|'b'|'d'|'l', 'v': value}` under `kidscribe_sync/{uid}`, so a number never returns as text
Service → ML Kit	An Ink object of timed points	strokes of `(x, y, t)`; coordinates are multiplied back to pixels at this boundary only

6.3 End-to-End Trace: a Child Checks the Letter A

The following is the complete path of one feature request through all three layers.

#	Layer	What happens
1	Presentation	The child traces with the stylus. `WritingPad` accumulates the points in a local controller; at roughly 60 points per second nothing is sent to the bloc.
2	Presentation	The child presses Check. The screen calls `cubit.checkGuided(_gPad.strokes)` and takes no decision of its own.
3	Business	`LetterPracticeCubit` calls `TraceEvaluator.evaluate(drawn:, template:)`.
4	Data	The evaluator computes coverage and precision, derives a score, then runs `checkOrder`, and returns `TraceResult(score: 88, stars: 3, passed: true, coverage: 0.96, precision: 0.91)`.
5	Business	Because the attempt passed, the cubit calls `ProgressService.saveGuided('A', 3)`, which keeps the better of the new and stored score.
6	Data	The service writes JSON to the key `letters_v2_<scope>` and appends to the practice log that drives the streak.
7	Business	`BadgeService.claimNewBadges()` returns any badge earned by this attempt, each exactly once.
8	Business	The cubit emits a new state: `emit(state.copyWith(guidedStars: 3, guidedResult: r, guidedTick: +1))`.
9	Presentation	`BlocBuilder` redraws the stars. A `BlocListener` watching `guidedTick` opens the result dialogue exactly once, which is how a one-shot event is distinguished from a redraw.

The presentation layer appears in only three of the nine steps, and in none of them does it decide anything. That is the architecture working as intended.

6.4 Division of Classes and Methods, with Code Metrics (CO1, CO3)

Four rules decided where each class boundary was placed: **single responsibility** (one class answers one question), **layering** (a class may only call the layer below), **reuse** (anything needed twice becomes its own class), and **testability by construction** (every bloc receives its services as constructor arguments, so a test can supply its own).

Metrics were measured over `lib/` with a brace-matching parser that strips comments and string literals first, so braces inside them cannot distort the counts. Generated folders are excluded.

Measure	Classes	Methods
Count	240	700
Maximum lines	**680** (`_HomeView`, home_screen.dart)	**263** (`_PennyPainter.paint`, mascot.dart)
Average lines	**71.9**	**17.1**
Median lines	37	7

Justification of the largest units. The median method is 7 lines, so the great majority of the code is in small units. Every method above about 130 lines is either a Flutter `build()` or a `CustomPainter.paint()`. A `build()` is a declarative widget tree with a single path through it rather than branching logic; splitting it into helpers relocates lines without reducing cyclomatic complexity and makes the layout harder to read. `_PennyPainter.paint` draws the mascot with sequential canvas calls and no branching. By contrast the classes that hold actual logic are deliberately small: `SentenceChecker` is 118 lines, `PasswordRules` 37, and `current_user.dart` 19. `TraceEvaluator`, the one substantial algorithm, is split into `evaluate()` and `checkOrder()` so each can be tested separately. The size therefore sits in drawing code, not in rules, which is where we intended it.

6.5 Implemented Features

Area	Delivered
Learning stages	Stage 1 strokes; Stage 2 all 26 letters, each with a guided half and an unguided half; Stage 3 whole words; Stage 4 sentences with word order, capitals and full stop; Stage 5 the parent hub.

Area	Delivered
Marking	Coverage and precision scoring with stars, designated stroke-order and direction checking with three distinct diagnostic messages, and ML-based free-hand recognition blended with geometry.
Games	Letter Match (five levels, look-alike letters from level three), Letter Ninja (timed), Free Writing.
Rewards	18 badges computed from real progress, and a daily streak that resets after a full missed day.
Parent tools	Child profiles, weekly progress with weak letters, exportable report file, daily reminder, writing challenges with time limits, toy-photo words with a blurry-photo check, demo premium, cloud sync, advanced report.
Teacher tools	Classes and students, per-student progress with a 'needs help' marker, classroom practice mode, homework and timed tests, grading with an answer key, written feedback, exportable reports.

6.6 Refactoring Carried Out After the Implementation Session

Refactor	Before	After and why
Testability seam for Firebase	Four services called `FirebaseAuth.instance` directly, which throws before Firebase is initialised, so none of them could be unit tested.	A 19-line `current_user.dart` wraps the call and falls back to a guest identity. Four services became testable with no change to production behaviour, and the application no longer crashes if a service is touched too early at start-up.
Password rules extracted	The rules were written twice inside the sign-up screen, once for the live checklist and once for the form validator, so the two could drift apart.	A single `PasswordRules` class now serves the screen and the tests. Tests T_7 to T_11 call it directly with no screen involved.
Shared widget set	Page scaffolding, cards and buttons were repeated in each screen.	`KidPage`, `KidHeader`, `KidCard` and `KidButton` are used by every screen, so a visual change is made once.
Writing pad unified	Each practice screen implemented its own pointer handling and palm rejection.	One `WritingPad` widget is shared by all practice screens, so palm rejection exists in exactly one place.

7. Tools, Version Control, Testing, and CI/CD

7.1 Selected Tools and the Alternatives Considered (CO4)

Area	Selected	Alternative	Justification and known limitation
Framework	Flutter 3.x / Dart 3.x	Native Android (Kotlin), React Native	Flutter gives a single codebase, a high-performance canvas suitable for stylus drawing, and hot reload. *Limitation:* larger install size than native, and plugin quality varies.
State management	flutter_bloc 8.1.6	Provider, Riverpod, setState	Enforces a visible separation between rules and interface and is testable without rendering. *Limitation:* more boilerplate; 29 bloc files for 30 screens.
Local storage	shared_preferences 2.2.3	sqflite, Hive, Isar	Our data is a few small documents read whole. *Limitation:* no queries, and everything must be serialised to a string.
Handwriting	google_mlkit_digital_ink_recognition	Cloud Vision or a custom model	Runs on device, so it is free per use and works offline. *Limitation:* a 20 MB one-off download, Android only, and untestable on a desktop runner.
Authentication	firebase_auth 5.3.1	A hand-written JWT service	Firebase issues and refreshes signed tokens, so the application never stores a password. *Limitation:* vendor lock-in and a Google dependency.
Sync	cloud_firestore 5.6.0	A REST API on our own server	No server to operate or pay for, and offline support is built in. *Limitation:* the free tier has quotas, and the data model must suit document storage.

7.2 Version Control Workflow

The project is on GitHub at `github.com/Kabirohi/kidscribe-spl2-project`. The repository holds **22 commits** on `main`, each scoped to one coherent piece of work and carrying a subject line and a body explaining the reasoning and the requirement it serves.

Because the three of us worked on one shared codebase, we adopted a **sequential integration protocol** rather than long-lived branches: one member at a time staged only the paths belonging to their area, committed, and pushed, and the next member pulled before starting. With clearly divided ownership this gave us a clean history and no merge conflicts across the whole integration. Each commit is authored by the member who wrote that code, so the contribution history on GitHub reflects the real division of work.

7.3 Testing Approach and Results

Our SRS defines 80 unit tests. We implemented an automated suite for every test that can be decided without a physical device, and a written manual procedure for the rest.

Measure	Result
Automated test cases written	**129**, in 15 files under `unit_testing/test`
SRS unit tests covered automatically	**58 of 80**
SRS unit tests run manually on the emulator	22 of 80, with written steps for each
Result of the last full run	**129 passed, 0 failed**, in about 23 seconds
Static analysis	`flutter analyze` — **no issues found**

The suite is run with a single command, `flutter test unit_testing/test`, and needs no emulator. It covers the marking algorithm and its stroke-order diagnosis, the Stage 4 grammar rules, stage unlocking, progress storage and learner separation, deadlines and timed tests, badges and streaks, the blurry-photo check, the premium lock, and report generation.

The 22 manual tests are those that genuinely require hardware: Firebase sign-up and sign-in (T_1 to T_6, T_12 to T_16), the handwriting model (T_29, T_30, T_39 to T_42), Android notifications (T_45, T_46), the camera (T_47), and two-device synchronisation (T_63, T_64). Their procedures are recorded in `unit_testing/KidScribe_Unit_Testing.pdf`.

An honest note on coverage. The automated tests exercise the decision logic directly rather than driving the interface. Where an SRS test is phrased as a user action — for example T_23, 'draw a straight line on the stroke screen' — the automated test calls the evaluator with a generated stroke. The decision is verified; the screen is not. Widget tests would close that gap and are listed in Section 11 as future work.

7.4 Continuous Integration and Delivery

CI/CD is planned but not implemented, and we would rather state that than overclaim. Quality gates were run locally before each push: `flutter analyze` and the full test suite. The planned pipeline is a GitHub Actions workflow triggered on push and pull request, running `flutter pub get`, `flutter analyze`, `flutter test unit_testing/test`, and `flutter build apk --debug`, publishing the APK as a build artefact and blocking merges on a failing suite. The work is small — a single workflow file — and was deprioritised against completing the functional requirements before the deadline.

8. Professional, Ethical, and Legal Considerations

8.1 Dependency and Asset Licence Audit (CO4)

Component	Version	Licence	Obligation met
Flutter SDK, Dart	3.x	BSD-3-Clause	Notice retained
flutter_bloc, equatable	8.1.6 / 2.0.5	MIT	Permissive; no action required
shared_preferences, path_provider	2.2.3 / 2.1.4	BSD-3-Clause	Notice retained
firebase_core, firebase_auth, cloud_firestore	3.6.0 / 5.3.1 / 5.6.0	BSD-3-Clause / Apache-2.0	Notice retained; Google API Terms accepted
google_mlkit_digital_ink_recognition, google_mlkit_image_labeling	0.14.1	MIT (plugin), Google ML Kit Terms (model)	Model terms accepted; on-device use only
confetti, audioplayers, image_picker, image, path_drawing, timezone	various	MIT / BSD	Permissive; no action required
flutter_local_notifications	17.2.3	BSD-3-Clause	Notice retained
google_fonts (Fredoka)	6.2.1	SIL Open Font License 1.1	Font licence permits embedding and redistribution
Artwork: Microsoft Fluent Emoji	120 files	**MIT**	**Licence text kept verbatim at `assets/art/LICENSE.txt` with attribution to Microsoft**
Success chime	1 file	Generated by the team	Original work; no third-party rights

Every dependency is permissively licensed and compatible with distributing the application. No copyleft licence that would require releasing our source under the same terms is present.

8.2 Privacy and the Protection of Children's Data

Our users include children under eight, which raises the standard of care. Four decisions follow from that.

- **Children hold no accounts and no credentials.** Only a parent or teacher registers. We collect no name, age, photograph or any other identifier directly from a child; a parent enters a first name and an age for their own records.
- **Everything is local by default.** All practice, scores and reports are stored on the device. Nothing leaves the tablet unless a parent explicitly turns on synchronisation.
- **No analytics, advertising, or third-party tracking** of any kind is present in the application, and no data is sold or shared.

- **Handwriting never leaves the device.** Recognition runs locally, so a child's writing is never transmitted to a server. This was a privacy decision as much as an offline one.

8.3 Security Measures

Passwords are never stored or handled by the application; Firebase Authentication holds the credential and returns a signed, automatically refreshed token. Synchronised data is protected by a Firestore security rule restricting every document to its owner:

```
match /kidscribe_sync/{uid} {
  allow read, write: if request.auth != null && request.auth.uid == uid;
}
```

The Firebase Android API key is present in the repository. This is expected and is not a secret: the key is embedded in every distributed Android package, and access is controlled by the package name, the signing certificate, and the security rule above rather than by hiding the key. Developer-machine configuration (`local.properties`) and signing material are excluded from version control.

8.4 Professional and Ethical Conduct

Two points deserve to be stated plainly. First, the **premium plan is a demonstration**: it asks for no card details, transmits nothing, and moves no money, and the interface labels it as a demonstration in the price text itself. A production release would use Google Play Billing, with the surrounding lock-and-unlock logic unchanged. Second, the application makes no clinical or diagnostic claim. It reports what a child has practised and which letters are weak; it does not assess dysgraphia or any other condition, and the reports are written to avoid implying that it does.

9. Project Management and Feasibility

9.1 Planned Milestones Against Actual Completion (CO7)

Session / milestone	Planned	Actual	Status
Project proposal and stakeholder research	June 2026	June 2026	On time
SRS session: requirements, user stories, test plan	July 2026	July 2026	On time
Design session: architecture, models, design report	July 2026	July 2026	On time
Stage 1 and 2 (strokes and letters) implemented	August 2026	July 2026	Early
Stage 3 and 4, games, rewards	September 2026	September 2026	On time
Parent and teacher tools, premium, sync	September 2026	September 2026	On time
Migration to BLoC architecture	Not originally planned	September 2026	Added after design feedback
Automated test suite (129 cases)	October 2026	October 2026	On time
Visual redesign, artwork and mascot	Not originally planned	October 2026	Added for the final demonstration
Final presentation	28 October 2026	28 October 2026	On time
Final report and repository submission	1 November 2026	1 November 2026	On time

9.2 Evidence of Iterative Planning and Task Tracking

We did not use a dedicated issue tracker such as Jira or Trello. Planning was carried out through short written plans agreed at the start of each phase, and tracked through the repository itself: each unit of work became a commit with a descriptive subject and a body naming the requirement it serves. The commit history is therefore our task record, and it is inspectable — the 22 commits group into the feature areas each member owned, in the order the application was built, from project setup through the five stages to the test suite and documentation. In hindsight a lightweight issue board would have made dependencies between our areas more visible, and we record that as a process lesson rather than claiming a tool we did not use.

9.3 Scope Changes and How They Were Handled

Change	Decision and communication
Design feedback asked for an explicit architecture.	We agreed to pause new features and migrate the whole application to BLoC before continuing, accepting a delay to feature work in exchange for a codebase that could be tested. The decision was recorded in ARCHITECTURE.md so all three of us applied it consistently.
The SRS grew by three functional requirements after review (FR-22, FR-23, FR-24).	Accepted in full and scheduled immediately, because all three were small and served existing stakeholders. The SRS matrix, the tests and the implementation were updated together so they could not drift.
A visual redesign with custom artwork was proposed late, close to the presentation.	Accepted on the condition that it changed no logic, so the test suite would still prove the application correct. After the redesign the full suite was re-run: 129 passed, analyser clean.
Suggestions to give children their own logins.	Rejected, because it contradicts FR-1 and our privacy position. Recorded in the SRS as out of scope with the reasoning, rather than left as an open question.

9.4 Feasibility, Effort and Cost Estimates (CO7)

The solution was feasible within the available time and resources, and the figures below are reasoned estimates rather than measured accounting.

Item	Estimate	Basis
Team effort	approximately 390-450 person-hours across five months	Three members, averaging 6-8 hours per week each over roughly 20 active weeks, with heavier weeks around each session deadline.
Development hardware	no additional cost	Existing personal laptops and one Android emulator; a physical tablet was borrowed for stylus testing.
Firebase (Authentication + Firestore)	0 BDT at current usage	The Spark free tier covers 50,000 authentication verifications and 50,000 document reads per day; a class of 40 children synchronising daily uses a tiny fraction of that.
Machine learning inference	0 BDT, permanently	Recognition runs on the device. A cloud recognition API would have cost per call and scaled with every letter every child writes — a recurring cost that the chosen design removes entirely.
Artwork and fonts	0 BDT	Microsoft Fluent Emoji (MIT) and Fredoka (SIL OFL).
Projected cost at scale	the first real cost appears in the low thousands of daily active users	Only synchronisation touches a paid service; everything else is local. Growth beyond the free tier would move Firestore to the pay-as-you-go tier at a few US dollars per month at that scale.

The principal technical risk — that on-device handwriting recognition would be too inaccurate to be useful — was retired early by prototyping it before the rest of Stage 2 was built, and mitigated by blending the model's judgement with our own geometric score so that the application degrades gracefully if the model is unavailable.

10. Teamwork and Individual Contributions

10.1 Responsibilities and Version-Control Traceability (CO5)

Work was divided by area of the application so that each member owned a coherent vertical slice and could be held accountable for it. Every commit is authored by the member who wrote the code.

Member (ID)	Area owned	Files	Commits	Representative commits
Zuhaaer Mustafa Sadab (230042132)	Visual identity and the way in: project setup, theme, artwork, shared widgets, the writing pad, authentication and sign-up	21 Dart files, ~3,800 lines, plus 120 art assets	6	Set up the Flutter project for Android; Add the app theme and colour palette; Add the shared screen pieces and the writing pad; Add login and sign up with Firebase
Md Shahib Al Kabir (230042157)	The child's learning journey: letter templates, progress storage, the marking algorithms, Stages 1 to 4, and the whole automated test suite	28 Dart files, ~5,700 lines, plus 15 test files	11	Add the data models and letter templates; Add Stage 1 and Stage 2: strokes and letters; Add Stage 4: building sentences; Add the unit tests
Hasan Mahi (230042129)	Games, rewards and the grown-up side: the three games, badges, child profiles and classes, the home screen, tasks and timed tests, reminders, toy words, premium, sync and reports	44 Dart files, ~7,600 lines	5	Add the three games; Add badges and the daily streak; Add child profiles, classes and the home screen; Add tasks, timed tests, feedback and reminders

10.2 Coordination of Interdependent Modules

Three interfaces crossed ownership boundaries, and each was agreed before the dependent work began.

- **The shared widget set.** Sadab's `KidPage`, `KidCard` and `KidButton` are used by every screen Kabir and Mahi wrote. The contract was that these widgets take content and never contain logic, which let all three of us build screens in parallel against a stable interface.
- **The writing pad.** Sadab built `WritingPad` and its palm rejection; Kabir's four practice screens consume it. We agreed the controller interface — finished strokes plus timing — before either side was written, so the ML integration and the pad could proceed independently.
- **Progress storage and the scope rule.** Kabir's `ProgressService` defines how a learner's data is keyed; Mahi's badges, reports, class dashboard and sync all read through it. Because the scope rule lives in one function, adding classroom mode later required no change in any consumer.

10.3 Handling Disagreements and Blockers

Two disagreements and one significant blocker are worth recording.

Issue	How it was resolved
Whether to migrate to BLoC mid-project. One view was that the existing code worked and the remaining time was better spent on features; the other was that untestable code would cost us more later.	We settled it on evidence rather than preference: we migrated two screens first and attempted to write tests for them. The tests were straightforward, which decided the question, and the full migration followed.
How to attribute commits during integration. Because we integrated from one machine at times, there was a question of whose name should appear.	We agreed that authorship must reflect who wrote the code, and used git's author field accordingly, leaving the committer field truthful about who ran the command. We rejected the alternative of sharing account credentials.
Blocker: the test suite could not run at all. Four services called Firebase directly, which throws before initialisation, so no service could be instantiated in a test.	Kabir identified the single shared cause and introduced a 19-line wrapper with a guest fallback. This unblocked the entire suite in one change and altered no production behaviour — the clearest instance in the project of a small refactor removing a large obstacle.

Participation was kept fair by dividing work into areas of comparable substance rather than by file count, and by requiring that each member be able to explain their own code without help — which the final presentation format demanded in any case.

11. Results, Limitations, and Future Work

11.1 Requirement Completion Status

Requirement group	Status	Evidence
FR-1 to FR-4 — accounts, profiles, classes	**Complete**	US-1 to US-7; T_1 to T_22
FR-5 to FR-9 — strokes, letters, feedback, words, sentences	**Complete**	US-8 to US-13; T_23 to T_34
FR-10 — five stages in order	**Complete**	US-14; T_35, T_36
FR-11 to FR-13 — the three games	**Complete**	US-15 to US-17; T_37 to T_42
FR-14, FR-15 — badges, streaks, reminders	**Complete**	US-18, US-19; T_43 to T_46
FR-16, FR-17 — toy words, parent challenges	**Complete**	US-20, US-21; T_47 to T_51
FR-18, FR-19 — class view, progress reports	**Complete**	US-22 to US-25; T_52 to T_60
FR-20 — offline storage and sync	**Complete**	US-26, US-27; T_61 to T_64
FR-21 — premium plan	**Complete as a demonstration**	US-28, US-29; T_65 to T_68. No real payment processing.
FR-22 to FR-24 — feedback, time limits, timed tests	**Complete**	US-30 to US-33; T_69 to T_80

All 33 user stories are implemented. Of the 80 unit tests, 58 are automated and passing; the remaining 22 require a physical device and are executed against a written procedure.

11.2 Known Limitations

- **Synchronisation never deletes.** The merge keeps the larger value and the union of collections, so no progress can be lost. The consequence is that a child profile or task deleted on one device reappears after the next sync. Fixing this needs deletion tombstones with timestamps, which we judged too risky to add close to the deadline.
- **The premium plan is a demonstration.** It is labelled as such in the interface and takes no payment details. Real billing requires Google Play Billing and a developer account.
- **22 of the 80 unit tests are manual.** They need Firebase, the ML model, Android notifications, a camera or two devices. Widget and integration tests would automate a further portion of them.
- **No continuous integration pipeline.** Quality gates are run locally and by convention rather than enforced automatically on push.
- **Storage does not support queries.** Comparing across many children requires loading each child's documents, which is acceptable for a class of forty but would not scale to a school.
- **Android only.** There is no iOS build or web version; the Firebase configuration and the ML plugin are configured for Android alone.
- **English capital letters only.** Lower case, digits and Bangla are not yet modelled, although the template format would accommodate them without structural change.

11.3 Future Work

Priority	Work	Why it matters
High	GitHub Actions pipeline running analyse, test and build on every push	Makes the quality gates we already use automatic and visible.
High	Widget and integration tests for the practice screens	Would automate a substantial part of the 22 manual tests.
High	Deletion tombstones in the sync payload	Removes the one behaviour in the product that can surprise a parent.
Medium	Lower-case letters, digits, and a Bangla letter set	The template format already supports it; this is content work plus recognition tuning.

Priority	Work	Why it matters
Medium	Spatial indexing of template points	Replaces the current all-pairs distance search; not needed at present sizes but the obvious next optimisation.
Low	A school-level dashboard across classes	Requires a queryable store, so it depends on migrating from key-value storage.

12. Glossary and References

12.1 Glossary

Term	Meaning
BLoC	Business Logic Component. A pattern that keeps application rules and state out of the interface; the screen listens for state and draws it.
Cubit	The simpler form of a BLoC, driven by method calls rather than named event objects.
Coverage (marking)	The fraction of the template shape that the child's ink came close to. Prevents a half-finished letter from passing.
Precision (marking)	The fraction of the child's ink that landed on the template. Prevents scribbling from passing.
Designated stroke order	The sequence and direction in which a letter is taught to be written. Checked separately from shape.
Normalised coordinates	Positions stored as fractions of the canvas between 0 and 1 rather than in pixels, so one set of shape data is correct on any screen.
Resampling	Rewriting a stroke as a fixed number of points spaced evenly along its length, so two strokes can be compared point for point.
Scope (storage)	The identifier that decides whose progress a stored value belongs to: a teacher's selected student, a parent's active child, or the account.
Palm rejection	Ignoring finger touches once a stylus has been detected, so a hand resting on the tablet does not draw.
Digital Ink Recognition	Reading handwriting from the sequence and timing of pen strokes rather than from a finished image.
SharedPreferences	The Android key-value store used for all local data. Holds only simple types, so maps are stored as JSON text.
FR / NFR / US	Functional requirement, non-functional requirement, and user story, as numbered in the SRS.

12.2 References

1. Group 18 (Mahi, Sadab, Kabir). *SWE 4404: SPL-II Project Proposal — KidScribe*, Department of CSE, IUT, June 2026.
2. Group 18. *KidScribe Software Requirements Specification*: 24 functional requirements, 33 user stories with acceptance criteria, and 80 unit tests. `/doc/srs.pdf`.
3. Group 18. *KidScribe Software Design Report*: three-layer architecture, class and data models, design alternatives. `/doc/design.pdf`.
4. Group 18. *KidScribe Final Presentation Deck*, presented 28 October 2026. `/doc/final-presentation.pdf`.
5. Group 18. *KidScribe Unit Testing Report*: how all 80 unit tests were run, with manual procedures. `/code/unit_testing/KidScribe_Unit_Testing.pdf`.
6. Group 18. *ARCHITECTURE.md*: the three layers, the Bloc-versus-Cubit decision, and the documented exception for pen input. Project repository.
7. KidScribe source repository. `https://github.com/Kabirohi/kidscribe-spl2-project`.
8. Department of Computer Science and Engineering, IUT. *Notice for SPL-II: Final Presentation*, 27 October 2026.
9. Google LLC. *Flutter Documentation*, https://docs.flutter.dev/.

10. Google LLC. *ML Kit Digital Ink Recognition Guide*, https://developers.google.com/ml-kit/vision/digital-ink-recognition.

11. Google LLC. *Cloud Firestore Security Rules Reference*, https://firebase.google.com/docs/firestore/security/get-started.

12. Felix Angelov and contributors. *flutter_bloc Package Documentation*, https://bloclibrary.dev/.

13. Microsoft Corporation. *Fluent Emoji* (MIT Licence), https://github.com/microsoft/fluentui-emoji.

14. Internet Engineering Task Force. *RFC 7519: JSON Web Token (JWT)*, May 2015.